

# AIFirst — Full Handover Documentation

**Audience:** non-technical team members and junior developers. **Purpose:** everything needed to take over the AIFirst website — content, code, deploy, troubleshooting.

This single file combines all chapters from the `docs/handover/` folder. A PDF version ( `HANDOVER_FULL.pdf` ) sits next to it.

## At a glance

| What                 | Where                                                               |
|----------------------|---------------------------------------------------------------------|
| Source code          | GitHub — <code>CrestAiLab/AIFirst</code>                            |
| Hosting              | Vercel                                                              |
| Content (CMS)        | Sanity (Studio at <code>/studio</code> )                            |
| Framework            | Next.js 15 + React 19 + TypeScript                                  |
| Styling              | Tailwind CSS + shadcn/ui                                            |
| Package manager      | pnpm 9+                                                             |
| Auto-deploy trigger  | Push to <code>main</code> branch on GitHub                          |
| Auto-refresh trigger | Editor publishes in Sanity → webhook → <code>/api/revalidate</code> |

## Table of contents

1. [Concepts & Glossary](#)
2. [Dev Environment Setup](#)
3. [Frontend](#)
4. [Backend](#)
5. [Sanity \(CMS\)](#)
6. [Vercel \(Hosting\)](#)
7. [Git & GitHub](#)
8. [CI/CD](#)
9. [How They Link Together](#)
10. [How To Deploy](#)
11. [How To Develop A New Feature](#)

# 00 — Concepts & Glossary

---

Audience: anyone new to web development. Read this first.

Every word in **bold** here is a word you will see again and again in the codebase, in Sanity, in Vercel, or in error messages. You don't need to memorise them — just know where to look them up.

---

## 1. The big picture in one paragraph

---

The AIFirst website is built with **Next.js** (a website framework). Editors write content inside **Sanity** (a content management system). The code is stored on **GitHub**. **Vercel** takes the code from GitHub, builds the website, and hosts it on the internet. When a content editor publishes something in Sanity, Sanity sends a **webhook** (a notification) to Vercel telling it "refresh the page now".

That's it. Everything else in this folder is detail.

---

## 2. The four moving parts

---

| Part                  | What it is                                 | Where it lives                              |
|-----------------------|--------------------------------------------|---------------------------------------------|
| <b>Code</b>           | The recipe for the website                 | GitHub repo <code>CrestAiLab/AIFirst</code> |
| <b>Content</b>        | Words and images shown on pages            | Sanity Studio ( <code>/studio</code> )      |
| <b>Hosting</b>        | The computer that serves the website       | Vercel                                      |
| <b>Build pipeline</b> | The robot that turns code into a live site | Vercel + Sanity webhooks                    |

---

## 3. Glossary

---

### Web basics

- **Frontend** — The part you see in the browser (buttons, text, images, layout).
- **Backend** — The hidden code that runs on a server. It fetches data, handles forms, talks to other services.
- **Server** — A computer somewhere on the internet that runs your code.
- **Client** — Your browser (Chrome, Safari) on your laptop or phone.
- **Browser** — The program you use to visit websites.
- **URL / route** — The address of a page (e.g. `/insights` , `/sources/my-article` ).

- **API** — A way for one program to ask another program for data. An **API route** in this project is a URL that returns data instead of a web page (e.g. `/api/revalidate` ).
- **HTTP request** — When the browser asks a server for something. Common methods: **GET** (fetch), **POST** (send/save).

## Code & tools

- **JavaScript** — The programming language web browsers understand.
- **TypeScript** — JavaScript with type safety (it warns you when you put a number where a string should go). Files end in `.ts` or `.tsx` .
- **React** — A library for building reusable pieces of UI called **components**.
- **Component** — A self-contained, reusable piece of UI (e.g. a button, a card, a header).
- **Next.js** — A framework built on top of React that handles pages, routing, server-side rendering, and API routes. We use **version 15** with the **App Router**.
- **App Router** — Next.js's modern way of defining pages: each folder under `/app` becomes a URL.
- **Tailwind CSS** — A way of styling pages by writing short **utility classes** (e.g. `text-lg font-bold text-white` ) directly in your HTML/JSX, instead of writing separate CSS files.
- **shadcn/ui** — A collection of pre-built, copy-pasteable components (button, card, input, tooltip) that use Tailwind. Lives in `components/ui/` .
- **Radix UI** — Low-level accessible UI primitives (we use it for tooltips). shadcn/ui is built on top of Radix.
- **Lucide** — An icon library. We use it for all icons ( `lucide-react` ).
- **Node.js** — A program that lets you run JavaScript outside a browser. Needed to run Next.js locally.
- **pnpm** — A package manager (like `npm` or `yarn` ). It downloads and installs the libraries the project needs. We require **pnpm 9 or higher**.
- **Package / dependency** — A library someone else wrote that we use. Listed in `package.json` .
- **Lockfile** ( `pnpm-lock.yaml` ) — Records the exact version of every dependency, so everyone gets the same install.
- **Build** — The process of turning source code into the optimised files that get served to users ( `pnpm build` ).
- **Dev server** — A local-only website that runs on your laptop while you're coding ( `pnpm dev` → `http://localhost:3000` ).

## Content management

- **CMS** — Content Management System. A friendly editor for non-developers to add/edit content without touching code.
- **Sanity** — The specific CMS we use. Two parts: **Sanity Studio** (the editor UI you visit at `/studio` ) and **Sanity Content Lake** (the cloud database where content is stored).
- **Schema** — The shape of a piece of content (which fields it has). Schemas live in `/sanity/schemaTypes/` .

- **Document** — A single saved item (one blog post, one source link, etc.).
- **Document type** — The kind of document. Our types are: `pageContent` (homepage), `insight` (blog post), `source` (curated link), `communityPost`, and `pageSection` (re-usable building block).
- **GROQ** — Sanity's query language. It looks like `*[_type == "insight"]`. You'll see GROQ in `lib/sanity/queries.ts`.
- **Portable Text** — Sanity's rich-text format (it's not raw HTML or Markdown — it's a structured tree). We render it with `@portabletext/react`.
- **Dataset** — A collection of documents inside a Sanity project. We use the dataset called `production`.
- **Project ID** — A short string that identifies our Sanity project (e.g. `abc123def`). Public — safe to expose.
- **API token** — A secret string used by code or scripts to write to Sanity. **Never share it publicly.**

## Hosting & deployment

- **Vercel** — The hosting platform we use. It takes our Next.js code and serves it to visitors worldwide.
- **Deployment** — A specific version of the website that has been built and pushed live. Vercel keeps a history of every deployment.
- **Production** — The version of the site that real users see (the `main` branch on Vercel).
- **Preview deployment** — A temporary version of the site Vercel creates for any other branch or pull request, so you can test before merging.
- **Environment variable** — A setting (or secret) loaded at runtime, e.g. `NEXT_PUBLIC_SANITY_PROJECT_ID`. Stored in `.env.local` for local dev and in Vercel's dashboard for production.
- **.env.local** — A local file (NOT committed to Git) containing your local environment variables.
- **Secret** — Any value you should not share publicly (API tokens, webhook secrets). Anything **without** the `NEXT_PUBLIC_` prefix is treated as a secret.
- **Webhook** — A "phone call" from one service to another when something happens. Sanity calls our `/api/revalidate` URL whenever an editor publishes content.
- **Revalidation** — Throwing away the old cached page and fetching the latest content. Done with Next.js's `revalidatePath()`.

## Code collaboration

- **Git** — The tool that tracks every change to the code over time.
- **GitHub** — The website where the Git repository is hosted.
- **Repository / repo** — A project's code + history. Ours: `CrestAiLab/AIFirst`.
- **Branch** — A parallel version of the code. We use `main` for production and `renovate` for the dependency bot.
- **Commit** — A saved snapshot of changes with a message describing them.

- **Push** — Upload your local commits to GitHub.
- **Pull / fetch** — Download the latest commits from GitHub.
- **Pull Request (PR)** — A proposal to merge a branch into another (usually into `main`). Reviewed before merging.
- **Merge** — Combine one branch into another.
- **Renovate** — A bot that opens PRs to update outdated dependencies.
- **CI/CD** — Continuous Integration / Continuous Deployment — short for "automatic checks and automatic deploy". Vercel does the CD part for us.

## Project-specific words

- **Section** — A horizontal block on the homepage (Hero, Stats, Solutions, etc.). Editors can reorder them in Sanity.
- **Insight** — A blog post / long-form article. Lives at `/insights/[slug]`.
- **Source** — A curated external link or piece of media (paper, podcast, article). Lives at `/sources`.
- **Community post** — A short discussion item shown on `/community` and (if `featured`) on the homepage.
- **Slug** — The URL-friendly version of a title. "My First Post" → `my-first-post`.
- **Default sections** — A hard-coded fallback homepage used when Sanity is unreachable, so the site never goes blank. See `lib/defaultSections.ts`.

---

## 4. Mental model: what happens when a visitor opens the site?

1. Visitor types `aifirst.example.com` → DNS sends them to Vercel.
2. Vercel has the latest **build** of the Next.js app ready and serves the page.
3. The page may include data fetched from Sanity (e.g. the latest 3 insights).
4. The browser displays the page.

## 5. Mental model: what happens when an editor publishes a new article?

1. Editor opens `/studio`, fills in fields, clicks **Publish**.
2. Sanity saves the document and sends a **webhook** to `https://oursite.com/api/revalidate`.
3. Our `/api/revalidate` route tells Next.js: "throw away cached `/insights` and `/`".
4. Next time someone visits, Next.js fetches the new content from Sanity and shows it.

## 6. Mental model: what happens when a developer pushes code?

1. Developer commits and pushes to GitHub.

2. GitHub notifies Vercel.
  3. Vercel runs `pnpm install && pnpm build`.
  4. If the build succeeds, Vercel switches `main` deployment to the new version.
  5. If it fails, the previous version stays live and Vercel emails the team.
- 

You now have the vocabulary. The next chapter will get the project running on your computer.

# 01 — Dev Environment Setup

Goal: get the website running on your own laptop in under 30 minutes.

## 1. What you need installed

| Tool                              | Why                                 | How to install                                                                                                         |
|-----------------------------------|-------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Git</b>                        | Download the code from GitHub       | macOS: comes pre-installed (run <code>git --version</code> ). Otherwise: <a href="https://git-scm.com">git-scm.com</a> |
| <b>Node.js</b> (v20 LTS or newer) | Runs JavaScript outside the browser | <a href="https://nodejs.org">nodejs.org</a> — download the LTS version                                                 |
| <b>pnpm</b> (v9 or newer)         | Installs the project's libraries    | After Node.js: <code>npm install -g pnpm</code>                                                                        |
| <b>Visual Studio Code</b>         | Recommended code editor             | <a href="https://code.visualstudio.com">code.visualstudio.com</a>                                                      |
| <b>GitHub account</b>             | To clone and push code              | <a href="https://github.com">github.com</a> — ask the team to add you to the <b>CrestAiLab</b> organisation            |

### Verify everything works

Open a terminal and run:

```
git --version      # → git version 2.x
node --version     # → v20.x or higher
pnpm --version     # → 9.x or higher
```

If any of these print "command not found", install that tool first.

## 2. Recommended VS Code extensions

Open VS Code → Extensions panel (left sidebar) → install:

- **ESLint** — flags code problems while you type
- **Tailwind CSS IntelliSense** — autocompletes Tailwind class names
- **Prettier** (optional) — auto-formats code on save
- **GitLens** — shows who changed what and when

### 3. Clone the repository

---

```
# pick a folder where you keep code projects
cd ~/GitHub

# download the code
git clone git@github.com:CrestAiLab/AIFirst.git
# OR if you don't use SSH keys yet:
git clone https://github.com/CrestAiLab/AIFirst.git

cd AIFirst
```

If `git clone` says "Permission denied", you need to either set up an SSH key (see GitHub docs → "Adding a new SSH key") or use the HTTPS URL and log in with a Personal Access Token.

### 4. Install the dependencies

---

```
pnpm install
```

This takes 1–3 minutes the first time. It downloads everything listed in `package.json` into a `node_modules/` folder.

**Common error:** `ERR_PNPM_UNSUPPORTED_ENGINE` — your pnpm is too old. Run `npm install -g pnpm@latest`.

### 5. Set up environment variables

---

The website needs a few secrets to talk to Sanity. They are kept out of Git.

#### Step 5a — Copy the template

```
cp .env.local.example .env.local
```

This creates a new file `.env.local` (gitignored) with empty values.

#### Step 5b — Fill in the values

Open `.env.local` in your editor. You'll see:



```
NEXT_PUBLIC_SANITY_PROJECT_ID=  
NEXT_PUBLIC_SANITY_DATASET=production  
SANITY_API_TOKEN=  
SANITY_REVALIDATE_SECRET=
```

### Where to get the values:

| Variable                      | Where to find it                                                                                                                                                                                                          |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NEXT_PUBLIC_SANITY_PROJECT_ID | <b>Easiest:</b> Vercel → AIFirst project → Settings → Environment Variables → click the eye icon to reveal. <b>Or:</b> <a href="https://sanity.io/manage">sanity.io/manage</a> → AIFirst project → Project ID at the top. |
| NEXT_PUBLIC_SANITY_DATASET    | Almost always <code>production</code> . Same Vercel page.                                                                                                                                                                 |
| SANITY_API_TOKEN              | <b>Optional for local dev</b> if you only read content. Required if you run <code>pnpm seed:sanity</code> . Get from Sanity → Manage → API → Tokens. Use a <b>Read</b> token unless you need to write.                    |
| SANITY_REVALIDATE_SECRET      | Optional locally — only needed if you want to test the webhook. Copy from Vercel.                                                                                                                                         |

**Don't have access to Vercel yet?** Ask the team owner to invite you to the Vercel team, then to the Sanity project at [sanity.io/manage](https://sanity.io/manage).

**Reminder:** never commit `.env.local` to Git. It's in `.gitignore` for a reason.

## 6. Run the dev server

```
pnpm dev
```

You should see:

```
▲ Next.js 15.x  
- Local:    http://localhost:3000  
- Ready in 1.2s
```

Open <http://localhost:3000> in your browser. The homepage should load.

If **env vars are missing**, the homepage still loads — it falls back to the **default sections** defined in `lib/defaultSections.ts`. You'll see the layout but data-driven sections (Insights, Sources) will be empty.

## Visit Sanity Studio

Open <http://localhost:3000/studio>. You'll see Sanity's editor running inside your local site. Log in with the same Sanity account that has access to the project.

---

## 7. The four most useful commands

---

| Command                       | What it does                                                                |
|-------------------------------|-----------------------------------------------------------------------------|
| <code>pnpm dev</code>         | Start the local dev server (live reload as you save)                        |
| <code>pnpm build</code>       | Build the production bundle — run this before pushing if you changed config |
| <code>pnpm start</code>       | Run the production build locally (mainly to debug build issues)             |
| <code>pnpm lint</code>        | Run ESLint to catch code style problems                                     |
| <code>pnpm seed:sanity</code> | Push the default homepage sections into Sanity (run once per new dataset)   |

---

## 8. Folder map cheat sheet

---

```
AIFirst/
├─ app/           ← Pages & API routes (every folder = a URL)
├─ components/    ← Reusable React UI pieces
├─ lib/           ← Utilities + Sanity queries + types
├─ sanity/        ← CMS schema definitions
├─ public/        ← Static assets (images, icons)
├─ scripts/       ← One-off scripts you run from the terminal
├─ docs/          ← Documentation (you are here)
└─ package.json   ← Project metadata + dependency list
```

---

## 9. Troubleshooting common errors

---

| Error                                                   | Likely cause                                                                     | Fix                                                                                 |
|---------------------------------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Cannot find module 'next'                               | <code>pnpm install</code> was skipped or failed                                  | Re-run <code>pnpm install</code>                                                    |
| Port 3000 is already in use                             | Another app is on port 3000                                                      | <code>pnpm dev -- -p 3001</code> to use a different port, or kill the other process |
| Configuration must contain 'projectId'                  | <code>NEXT_PUBLIC_SANITY_PROJECT_ID</code> is missing in <code>.env.local</code> | Add it (see step 5)                                                                 |
| <code>/studio</code> shows blank screen                 | Browser cache, or you're not logged into Sanity                                  | Hard refresh (Cmd+Shift+R), then log in                                             |
| Images broken in dev                                    | Sanity CORS doesn't allow <code>localhost:3000</code>                            | Add it in Sanity Manage → API → CORS Origins                                        |
| EACCES on macOS during <code>npm install -g pnpm</code> | Permission issue with global install                                             | Use <code>sudo</code> , or use <a href="#">Volta</a> to manage Node                 |

For deeper diagnosis: visit <http://localhost:3000/api/check-setup> — it returns a JSON report of what's configured and what's missing.

---

## 10. You're set up. What now?

---

- Read [02 — Frontend](#) to learn the codebase structure.
- Read [10 — How To Develop A New Feature](#) when you're ready to make a change.

# 02 — Frontend

---

Audience: junior developers. The "frontend" is everything a visitor sees in their browser.

---

## 1. What runs the frontend?

---

- **Next.js 15** (App Router) — handles routing and rendering.
- **React 19** — builds the UI out of components.
- **TypeScript** — adds type-safety on top of JavaScript.
- **Tailwind CSS** — styling via short utility classes ( `text-lg font-bold` ).
- **shadcn/ui + Radix UI + lucide-react** — pre-built buttons, cards, tooltips, icons.

You don't need to know any of these in depth before contributing — copy an existing example and tweak it.

---

## 2. Where things live

---

```
app/           ← Pages (one folder per URL)
components/    ← Reusable building blocks
  └─ ui/        ← Generic primitives (Button, Card, Input, Tooltip)
lib/           ← Helper code (queries, utils, types)
  └─ sanity/    ← Talking to the Sanity CMS
public/        ← Static images and icons
```

## 3. The pages (URLs)

---

Every folder under `app/` with a `page.tsx` file becomes a URL. Here is the full list:

| URL              | File                           | What it shows                                                                                    |
|------------------|--------------------------------|--------------------------------------------------------------------------------------------------|
| /                | app/page.tsx                   | Homepage — renders sections fetched from Sanity (Hero, Stats, Solutions, Insights preview, etc.) |
| /insights        | app/insights/page.tsx          | All blog posts, paginated 10 per page                                                            |
| /insights/[slug] | app/insights/[slug]/page.tsx   | A single blog post (rich text)                                                                   |
| /sources         | app/sources/page.tsx           | All curated links (papers, podcasts, articles), paginated                                        |
| /community       | app/community/page.tsx         | Community posts/discussions, paginated                                                           |
| /contact         | app/contact/page.tsx           | Contact info                                                                                     |
| /studio          | app/studio/[...index]/page.tsx | Sanity Studio (the editor) embedded in the site                                                  |

The square brackets `[slug]` mean **dynamic route** — the value in the URL is passed as a parameter (e.g. `/insights/why-ai-matters` → `slug = "why-ai-matters"` ).

The double square brackets `[...index]` mean **catch-all** route — Sanity Studio uses internal routing.

## 4. The root layout

`app/layout.tsx` wraps every page. It is the HTML skeleton: `<html>` , `<body>` , fonts, providers, analytics, header, footer.

If you want a change to apply to **every** page (e.g. a new font, a global notification banner), edit `layout.tsx` .

## 5. Components ( /components )

Components are reusable pieces of UI. Each is a `.tsx` file that exports a React function.

### Layout chrome

- `header.tsx` — top navigation
- `footer.tsx` — page footer
- `app-tooltip-provider.tsx` — wires up tooltips globally

## Homepage section components

The homepage is a list of "sections" (Hero, Stats, Solutions, etc.) configured in Sanity. `section-renderer.tsx` reads each section's `sectionType` and renders the matching component:

- `hero.tsx`
- `stats.tsx`
- `solutions.tsx`
- `cyber-security.tsx`
- `community.tsx`
- `insights.tsx`
- `sources.tsx`
- `cta.tsx`
- `content.tsx`

## Listing helpers

- `source-card.tsx` — card for a single source on the listing
- `source-description-markdown.tsx` — renders Markdown descriptions
- `list-pagination.tsx` — Prev / Page numbers / Next controls
- `show-more-button.tsx` — the "Explore more →" link

## UI primitives ( `/components/ui` )

Generic, project-agnostic components from shadcn/ui:

- `button.tsx` , `card.tsx` , `input.tsx` , `tooltip.tsx`

**Rule of thumb:** if a piece of UI is used in more than one place, make it a component.

---

## 6. Styling with Tailwind

Tailwind lets you style by adding class names directly:

```
<button className="px-4 py-2 rounded-lg bg-white text-black hover:bg-gray-200">
  Click me
</button>
```

That single line gives you padding, rounded corners, white background, dark text, and a hover state. No separate CSS file needed.

## Useful conventions in this project

- **Dark mode** is on by default. We use `bg-black / text-white` as the base.
- **Responsive prefixes:** `sm:` ( $\geq 640\text{px}$ ), `md:` ( $\geq 768\text{px}$ ), `lg:` ( $\geq 1024\text{px}$ ). Example: `text-base md:text-lg`.
- **`cn()` helper** in `lib/utils.ts` merges class names safely. Use it when conditionally applying classes:

```
className={cn("p-4 rounded", isActive && "bg-blue-500")}
```

The full Tailwind reference: [tailwindcss.com/docs](https://tailwindcss.com/docs).

---

## 7. Where data comes from

Components don't talk to Sanity directly. They receive props from a page, and pages fetch data using helpers in `lib/sanity/`:

- `lib/sanity/client.ts` — the Sanity client object
- `lib/sanity/queries.ts` — all GROQ queries (one per use case)
- `lib/sanity/image.ts` — `urlFor(image)` to build optimised image URLs
- `lib/sanity/types.ts` — TypeScript types for everything Sanity returns

Example from `app/insights/page.tsx` (simplified):

```
import { client } from '@lib/sanity/client'
import { insightsPaginatedQuery } from '@lib/sanity/queries'

export default async function InsightsPage({ searchParams }) {
  const insights = await client.fetch(insightsPaginatedQuery, { start: 0, end: 10 })
  return <InsightsList items={insights} />
}
```

The `await client.fetch(...)` happens **on the server** during the request, so the visitor gets a fully-rendered page.

---

## 8. Server vs Client components

Next.js App Router defaults to **Server Components** — they run on the server, can fetch data directly, and never ship JavaScript to the browser. Lighter, faster.

If a component needs **interactivity** (state, click handlers, hooks like `useState` / `useEffect`), it must be a **Client Component**. Mark it by adding `'use client'` at the very top of the file:

```
'use client'
import { useState } from 'react'

export function Counter() {
  const [count, setCount] = useState(0)
  return <button onClick={() => setCount(count + 1)}>{count}</button>
}
```

**Rule of thumb:** start with a server component. Only convert to a client component when you need interactivity.

---

## 9. Adding a new page

---

To add a page at `/about` :

1. Create the folder: `app/about/` .
2. Create `app/about/page.tsx` :

```
export default function AboutPage() {
  return (
    <main className="container mx-auto px-4 py-16">
      <h1 className="text-4xl font-bold">About us</h1>
      <p className="mt-4 text-lg">We build the AIFirst platform...</p>
    </main>
  )
}
```

3. Save. Visit `http://localhost:3000/about` — done.

To add a navigation link to the new page, edit `components/header.tsx` .

---

## 10. Adding a new component

---

1. Create the file: `components/my-new-card.tsx` .
2. Define a function that returns JSX:

```
type Props = { title: string; description: string }

export function MyNewCard({ title, description }: Props) {
  return (
    <div className="rounded-xl border border-white/10 p-6">
      <h3 className="text-xl font-semibold">{title}</h3>
      <p className="mt-2 text-white/70">{description}</p>
    </div>
  )
}
```



3. Import it where you need it: `import { MyNewCard } from '@components/my-new-card'` .

The `@/` prefix is an alias for the project root (configured in `tsconfig.json` ).

---

## 11. Images

---

- **Static images** (decorations, backgrounds) → put in `public/` and reference as `/my-image.jpg` .
- **Sanity images** (uploaded by editors) → use `urlFor(image).width(800).url()` from `lib/sanity/image.ts` .
- **External images** (e.g. og:image of a source URL) → fetched via the proxy at `/api/sources/thumbnail` to avoid hotlink blocks.

Always pair an image with descriptive `alt` text for accessibility.

---

## 12. Common gotchas

---

- **Forgot 'use client'** → error about hooks not allowed in server components. Add `'use client'` at the top of the file.
  - **Edited a Sanity schema, page still shows old fields** → restart the dev server.
  - **Tailwind class doesn't take effect** → make sure the file path is in `tailwind.config.ts` `content` array, and that the class is a literal string (Tailwind can't see `text-${size}` dynamic strings).
  - **Image loads slowly** → use `next/image` (the `<Image>` component) which auto-optimises.
- 

Next: see [03 — Backend](#) for the API routes, or [04 — Sanity](#) for content schemas.

## 03 — Backend

---

The "backend" is the code that runs on Vercel's servers, not in the visitor's browser. In Next.js this is mostly **API routes** plus the data-fetching that happens inside server components.

---

### 1. What counts as backend in this project?

---

We don't run a separate backend. Next.js does both jobs:

| Backend job                                | Where it happens                                                                         |
|--------------------------------------------|------------------------------------------------------------------------------------------|
| Render pages with fresh content            | Server Components (the default — see <a href="#">Frontend §8</a> )                       |
| Receive webhooks and trigger cache refresh | <code>/app/api/revalidate/route.ts</code>                                                |
| Health / diagnostic checks                 | <code>/app/api/check-setup/route.ts</code> , <code>/app/api/debug-sanity/route.ts</code> |
| Proxy external images for source cards     | <code>/app/api/sources/thumbnail/route.ts</code>                                         |

There is **no separate database** — all content lives in Sanity. There is **no auth system** — Studio uses Sanity's own login.

---

### 2. How API routes work in Next.js

---

Any file at `app/api/<name>/route.ts` becomes an API endpoint at `/api/<name>` . The file exports a function for each HTTP method ( `GET` , `POST` , etc.) it supports.

Minimal example:

```
// app/api/hello/route.ts
import { NextResponse } from 'next/server'

export async function GET() {
  return NextResponse.json({ message: 'hello world' })
}
```

That's it. Visit `http://localhost:3000/api/hello` and you get `{"message":"hello world"}` .

---

### 3. The four API routes in this project

---

#### POST | GET /api/revalidate

**Purpose:** receive Sanity's webhook and tell Next.js to refresh cached pages.

**How it's triggered:** Sanity calls this URL every time an editor publishes, updates, or deletes a document.

**Security:** the URL must include `?secret=...` matching `SANITY_REVALIDATE_SECRET`. If the secret is wrong, the request is rejected.

**What it revalidates:** depending on the document type that changed:

- `pageContent` → `/`
- `insight` → `/insights`, `/insights/[slug]`, `/`
- `source` → `/sources`, `/`
- `communityPost` → `/community`, `/`

**Manual test:** open `https://yoursite.com/api/revalidate?secret=THE_SECRET` in your browser. You should see a JSON success response.

#### GET /api/check-setup

**Purpose:** a friendly diagnostic page. Visit it whenever the site looks broken.

**What it returns:**

- whether each environment variable is set,
- whether we can reach Sanity,
- a list of recommendations.

**Safe to share** — secret values are not printed.

#### GET /api/debug-sanity

**Purpose:** runs a small Sanity query and returns the result. Used to verify the API token works.

**Note:** consider removing this endpoint (or putting it behind a check) before going live with sensitive content. Right now it is fine because we only have public content.

#### GET /api/sources/thumbnail?img=<url>

**Purpose:** fetch an external image (the `og:image` of a source URL) on the server and return it to the browser. Avoids browser hotlink/CORS blocks.

**Why we need it:** when a source has only a URL but no uploaded thumbnail, the source card tries to use the page's `og:image`. Many sites refuse direct browser requests, but allow server-to-server fetches.

**Behaviour:** returns the image with a 1-day cache header, or a placeholder SVG if the fetch fails.

---

## 4. Where each route reads its data

---

| Route                               | Reads from                                                                                       |
|-------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>/api/revalidate</code>        | Reads the JSON body of the webhook from Sanity                                                   |
| <code>/api/check-setup</code>       | Reads env vars + makes a simple Sanity ping                                                      |
| <code>/api/debug-sanity</code>      | Runs a GROQ query via <code>lib/sanity/client.ts</code>                                          |
| <code>/api/sources/thumbnail</code> | Fetches an external HTTP URL (validated against an allowlist in <code>lib/url-safety.ts</code> ) |

---

## 5. Environment variables used by the backend

---

| Variable                                   | Used by                                                  | Public?              |
|--------------------------------------------|----------------------------------------------------------|----------------------|
| <code>NEXT_PUBLIC_SANITY_PROJECT_ID</code> | Sanity client                                            | Yes (safe to expose) |
| <code>NEXT_PUBLIC_SANITY_DATASET</code>    | Sanity client                                            | Yes                  |
| <code>SANITY_API_TOKEN</code>              | Server-only writes (e.g. <code>pnpm seed:sanity</code> ) | <b>No — secret</b>   |
| <code>SANITY_REVALIDATE_SECRET</code>      | <code>/api/revalidate</code> security check              | <b>No — secret</b>   |

**Rule:** anything starting with `NEXT_PUBLIC_` is shipped to the browser. Everything else stays on the server.

---

## 6. How webhooks flow end-to-end

---

```
Editor clicks "Publish" in /studio
  |
  ▼
Sanity sends POST https://yoursite.com/api/revalidate?secret=...
  |
  ▼
/app/api/revalidate/route.ts validates the secret
  |
  ▼
It calls revalidatePath('/insights') etc.
  |
  ▼
Next.js drops the cached pages
  |
  ▼
Next visitor triggers a fresh fetch from Sanity → sees updated content
```

If the webhook fails, the page eventually refreshes anyway thanks to time-based cache, but it can take longer (hours).

---

## 7. Adding a new API route

---

Goal: create `/api/health` that returns `{ ok: true }`.

1. Create the file `app/api/health/route.ts` :

```
import { NextResponse } from 'next/server'

export async function GET() {
  return NextResponse.json({ ok: true, time: new Date().toISOString() })
}
```

2. Save. Visit `http://localhost:3000/api/health` — you should see the JSON.
3. Commit and push — it will be live on Vercel automatically.

If your route reads request bodies, you'd add `POST`, `PUT`, etc. functions and read with `await req.json()`.

---

## 8. Server-side caching

---

Next.js caches `fetch()` results by default. To opt out (e.g. for an always-fresh route), add:

```
export const dynamic = 'force-dynamic'
// or
export const revalidate = 0
```

This is already handled in our existing routes; you only need it if you build something new that must always run fresh.

---

## 9. Logs & debugging

---

- **Locally:** `pnpm dev` shows logs in the terminal. Use `console.log()` freely.
  - **Production:** open Vercel → AIFirst project → Logs tab → filter by route. Each call to an API route appears with its status code and any console output.
- 

## 10. Common pitfalls

---

- **Webhook returns 401** → secret in the URL doesn't match `SANITY_REVALIDATE_SECRET`. Re-check both Sanity webhook config and Vercel env var.
  - **Image proxy returns placeholder** → the source URL is on a domain not in the allowlist ( `lib/url-safety.ts` ), or the upstream site refused the request. Add the domain or upload a thumbnail manually in Sanity.
  - `/api/check-setup` says **token missing** → set it in Vercel → Environment Variables → Redeploy.
- 

Next: [04 — Sanity](#) to understand the content side.

# 04 — Sanity (CMS)

Audience: editors and junior developers. Sanity is where the content lives. The website reads from it; the code is just the wrapper.

## 1. What Sanity is

Sanity has two parts:

| Part                       | What it is                                | Where it runs                                                  |
|----------------------------|-------------------------------------------|----------------------------------------------------------------|
| <b>Sanity Studio</b>       | The friendly editor UI                    | Embedded in our site at <code>/studio</code> (no separate URL) |
| <b>Sanity Content Lake</b> | The cloud database storing every document | sanity.io's servers                                            |

The website reads from Content Lake using **GROQ** queries. Editors write to Content Lake by clicking **Publish** in Studio.

## 2. Project identifiers

| Field                   | Value                                                     | Where to find                                                                                                 |
|-------------------------|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Project ID</b>       | A short opaque string (e.g. <code>abc123def</code> )      | <a href="https://sanity.io/manage">sanity.io/manage</a> → AIFirst → top-left, or copy from Vercel env vars    |
| <b>Dataset</b>          | <code>production</code>                                   | Same place                                                                                                    |
| <b>Studio URL</b>       | <code>/studio</code> on whatever site is serving the code | e.g. <code>http://localhost:3000/studio</code> in dev, <code>https://yoursite.com/studio</code> in production |
| <b>Manage dashboard</b> | <a href="https://sanity.io/manage">sanity.io/manage</a>   | Where you set CORS, webhooks, and tokens                                                                      |

## 3. Logging into Studio

1. Go to `/studio` .
2. Click **Sign in with Google** (or your team's chosen provider).
3. If you don't see content, your account isn't a member of the Sanity project — ask the project owner to invite you at `sanity.io/manage` → Members.

---

## 4. The content schemas (document types)

---

A schema defines what fields a document has. Schemas live in `/sanity/schemaTypes/`. There are five:

### 4.1 `pageContent` — the homepage configuration

Single document. The whole homepage layout is configured here.

| Field                 | What it does                                                                                             |
|-----------------------|----------------------------------------------------------------------------------------------------------|
| <code>title</code>    | Internal name (not shown on the site)                                                                    |
| <code>sections</code> | An ordered list of <b>page sections</b> (Hero, Stats, Solutions, etc.) — drag to reorder, toggle to hide |

If this document doesn't exist yet, the site falls back to defaults from `lib/defaultSections.ts`.  
Run `pnpm seed:sanity` to create it.

### 4.2 `pageSection` — a building block (used inside `pageContent`)

Not a standalone document. Each section has a `sectionType` field that controls which other fields appear:

| <code>sectionType</code>   | Component on the homepage   |
|----------------------------|-----------------------------|
| <code>hero</code>          | Big headline + 2 buttons    |
| <code>stats</code>         | Row of metrics              |
| <code>solutions</code>     | Bento grid of feature cards |
| <code>cyberSecurity</code> | Security hub block          |
| <code>community</code>     | Community pathways block    |
| <code>insights</code>      | Latest 3 blog posts         |
| <code>sources</code>       | Latest 3 curated links      |
| <code>cta</code>           | Call-to-action banner       |
| <code>content</code>       | 2-column text + image block |

Each section also has:

- `enabled` — toggle visibility
- `showMore` — optional "Explore more →" link (internal or external)



### 4.3 insight — a blog post

| Field       | Notes                                                                                 |
|-------------|---------------------------------------------------------------------------------------|
| title       | The headline                                                                          |
| slug        | Auto-generated from the title (becomes the URL: <code>/insights/&lt;slug&gt;</code> ) |
| category    | One of: Research, Case Study, Guide, News, Tutorial                                   |
| description | Short preview shown in cards                                                          |
| image       | Cover image (uploaded to Sanity)                                                      |
| publishedAt | Date — defaults to "now"                                                              |
| content     | Rich text + inline images (Portable Text editor)                                      |
| author      | Defaults to "AI First Team"                                                           |

Listed at `/insights` , individually at `/insights/[slug]` , last 3 on the homepage.

### 4.4 source — a curated external link

| Field       | Notes                                                                                   |
|-------------|-----------------------------------------------------------------------------------------|
| title       | Display title                                                                           |
| slug        | URL slug (used internally even if the card links externally)                            |
| kind        | <code>paper</code> , <code>podcast</code> , <code>article</code> , or <code>link</code> |
| url         | Where the card links to. If set, the card opens this URL.                               |
| sourceInfo  | "From The Verge", "MIT Press", etc.                                                     |
| tags        | Small chips on the card                                                                 |
| description | Markdown supported                                                                      |
| thumbnail   | Optional — overrides the auto-fetched og:image                                          |
| publishedAt | Date                                                                                    |

Listed at `/sources` , last 3 on the homepage. If `thumbnail` is empty and `url` is set, the site fetches the og:image via `/api/sources/thumbnail` .

## 4.5 communityPost — a discussion item

| Field     | Notes                                                     |
|-----------|-----------------------------------------------------------|
| title     | Headline                                                  |
| slug      | URL slug                                                  |
| author    | { name, avatar }                                          |
| content   | Plain text                                                |
| tags      | e.g. #RAG , #CyberSecurity                                |
| replies   | Count (defaults to 0)                                     |
| createdAt | Date                                                      |
| featured  | If true , shown on the homepage community section (max 3) |

Listed at /community .

---

## 5. Editor workflows

---

### Edit the homepage layout

1. Open /studio → **Page Content**.
2. Click on the **Sections** array. Drag to reorder, toggle enabled , or click a section to edit its fields.
3. Click **Publish** (bottom-right). Within a few seconds the homepage updates.

### Add a new blog post (insight)

1. /studio → **Insights** → **New Insight**.
2. Fill in title (slug auto-generates), description, upload a cover image, pick category, write content in the rich-text editor.
3. **Publish** → live at /insights/your-slug and on /insights listing.

### Add a curated source

1. /studio → **Sources** → **New Source**.
2. Choose kind , paste url , fill title , sourceInfo , add tags.
3. (Optional) upload a custom thumbnail . Otherwise the site grabs the page's preview image.
4. **Publish**.

### Add a community post

1. /studio → **Community Posts** → **New Community Post**.

2. Fill author name, optional avatar, content, tags.
3. Tick `featured` if you want it on the homepage.
4. **Publish.**

---

## 6. The webhook (so content updates show instantly)

---

When an editor publishes, Sanity calls `https://yoursite.com/api/revalidate?secret=...`. This drops Next.js's cache and the next visitor sees the new content.

### Setup (one-time, by an admin)

1. [sanity.io/manage](https://sanity.io/manage) → AIFirst → **API** → **Webhooks** → **Create webhook**.
2. Fill in:
  - **URL:** `https://yoursite.com/api/revalidate?secret=THE_SECRET`
  - **Trigger on:** Create, Update, Delete
  - **Filter:** leave empty (or restrict to specific document types)
  - **Projection:** `{ _type, _id, slug }`
3. The `THE_SECRET` value must equal Vercel's `SANITY_REVALIDATE_SECRET` env var. If you don't have one yet:

```
openssl rand -hex 32
```

Paste the result into both Sanity webhook URL and Vercel env vars.

### Verifying the webhook works

- Edit any insight → publish → watch the website refresh within ~5 seconds.
- Or check the Sanity webhook log (Manage → Webhooks → click the webhook → see recent deliveries).
- Or call manually: `https://yoursite.com/api/revalidate?secret=THE_SECRET` should return `{ "revalidated": true, ... }`.

---

## 7. CORS setup (so Studio works)

---

Sanity blocks browsers from talking to it unless your site's URL is on the **CORS Origins** list.

[sanity.io/manage](https://sanity.io/manage) → AIFirst → **API** → **CORS Origins** → add:

- `http://localhost:3000` (for local dev)
- `https://yoursite.vercel.app` (the Vercel production URL)
- any custom domain you use

Tick **Allow credentials** for each.

---

## 8. API tokens

---

For scripts that need to **write** to Sanity (e.g. `pnpm seed:sanity`):

1. [sanity.io/manage](https://sanity.io/manage) → AIFirst → **API** → **Tokens** → **Add API token**.
  2. Choose the right permission:
    - **Viewer** — read only (safe for fetches)
    - **Editor** — read + write (for seed scripts)
    - **Admin** — full control (avoid unless needed)
  3. Copy the token. **It is shown only once.**
  4. Paste into `.env.local` as `SANITY_API_TOKEN`, and into Vercel env vars if needed in production.
- 

## 9. The seed script

---

`pnpm seed:sanity` (file: `scripts/seed-sanity.ts`) creates the initial homepage configuration. Run it once when setting up a brand-new dataset. It:

- checks if a `pageContent` document exists,
- if not, creates one with the default sections from `lib/defaultSections.ts`,
- if yes, optionally updates it.

Other scripts in `/scripts/`:

- `init-homepage.ts` — interactive variant of seed
- `check-sanity-data.ts` — validates Sanity content is present and well-formed

See `scripts/README.md` for details.

---

## 10. Common questions

---

**Q: I published a change but the site still shows the old content.** A: Wait 5 seconds. Then check `/api/check-setup` and the Sanity webhook log. If the webhook isn't firing, see [WEBHOOK\\_TROUBLESHOOTING.md](#).

**Q: I deleted a document by accident.** A: Sanity keeps a history. Open the document in Studio → click the timeline (clock icon, top-right) → restore.

**Q: Can I have draft content that's not yet public?** A: Yes — Studio's "Save Draft" doesn't publish. Drafts aren't visible to the public site (which only reads published documents).

**Q: Where do I see all changes ever made to a document?** A: Same timeline (clock icon) in Studio.

**Q: Can two editors work at once?** A: Yes. Studio is real-time collaborative.

---

Next: [05 — Vercel](#) for the hosting side.

# 05 — Vercel (Hosting)

---

Vercel is the company that hosts the site on the internet. It also automatically rebuilds and redeploys the site every time we push code.

---

## 1. What Vercel does for us

---

| Job                   | How                                                                              |
|-----------------------|----------------------------------------------------------------------------------|
| Hosts the website     | Serves <code>https://aifirst...vercel.app</code> (or your custom domain)         |
| Builds the code       | Runs <code>pnpm install &amp;&amp; pnpm build</code> whenever we push to GitHub  |
| Auto-deploys          | If the build succeeds, the new version replaces the old one                      |
| Preview URLs          | Every branch / PR gets its own temporary URL for review                          |
| Environment variables | Stores secrets (Sanity tokens, etc.) so the code can read them at build/run time |
| Logs & analytics      | Shows requests, errors, and performance                                          |
| Domains               | Connects custom domains and handles SSL                                          |

You do **not** need to log into a server, install Node.js, or copy files. Push to GitHub → Vercel does the rest.

---

## 2. The Vercel dashboard

---

Go to [vercel.com](https://vercel.com) → log in → select the **AIFirst** project. The main tabs:

| Tab                              | What you do there                                                                                                                    |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Overview                         | See the latest production deployment + recent activity                                                                               |
| Deployments                      | History of every build (production and previews). Click any to inspect, view logs, or <b>rollback</b>                                |
| Settings → Environment Variables | Add / edit <code>NEXT_PUBLIC_SANITY_PROJECT_ID</code> , <code>SANITY_API_TOKEN</code> , <code>SANITY_REVALIDATE_SECRET</code> , etc. |
| Settings → Domains               | Connect a custom domain (e.g. <code>aifirst.com</code> ). Vercel handles SSL automatically                                           |
| Settings → Git                   | Which GitHub repo + branch is connected (currently <code>CrestAiLab/AIFirst</code> , branch <code>main</code> )                      |
| Logs (Runtime Logs)              | Live output of API routes and server components — useful for debugging production                                                    |
| Analytics                        | Traffic & Web Vitals (we use <code>@vercel/analytics</code> )                                                                        |

### 3. Environment variables

Vercel needs the same env vars as `.env.local` to talk to Sanity.

#### Add or edit a variable

1. Project → **Settings** → **Environment Variables**.
2. Type the name (e.g. `SANITY_API_TOKEN` ) and value.
3. Choose where it applies:
  - **Production** — the `main` branch
  - **Preview** — any other branch / PR
  - **Development** — when someone uses `vercel dev` locally (rare; we use `pnpm dev` )
4. Click **Save**.
5. **Important:** existing deployments won't pick up the new variable until you **redeploy**. Either push a new commit or click **Redeploy** on the latest deployment.

#### Naming rule

- `NEXT_PUBLIC_*` → safe to ship to the browser.
- Anything else → kept secret on the server.

## Required values

| Variable                                   | Required?                       | Notes                                      |
|--------------------------------------------|---------------------------------|--------------------------------------------|
| <code>NEXT_PUBLIC_SANITY_PROJECT_ID</code> | Yes                             | Public                                     |
| <code>NEXT_PUBLIC_SANITY_DATASET</code>    | Yes ( <code>production</code> ) | Public                                     |
| <code>SANITY_API_TOKEN</code>              | Optional                        | Needed if any server code writes to Sanity |
| <code>SANITY_REVALIDATE_SECRET</code>      | Strongly recommended            | Webhooks won't work without it             |

---

## 4. Deployments

Each push to GitHub creates a deployment.

### Production deployment (the live site)

- Triggered by a push to `main` .
- URL: the production domain (custom or `aifirst.vercel.app` ).
- A failed build keeps the previous version live — no broken site.

### Preview deployment (per branch / PR)

- Triggered by a push to any other branch or by opening a PR.
- URL: something like `aifirst-git-feature-x-crestailab.vercel.app` .
- Useful for sharing in-progress work without merging.

### Rollback

1. **Deployments** tab → find a previous successful production deployment.
2. Click the `...` menu → **Promote to Production**.
3. The site instantly serves that older build.

This is the safest "undo" for a bad deploy. You don't have to revert in Git.

---

## 5. Domains

To connect a custom domain (e.g. `aifirst.com` ):

1. **Settings** → **Domains** → **Add**.
2. Type the domain.



3. Vercel shows DNS instructions (an A record or CNAME). Add them at the domain registrar (GoDaddy, Cloudflare, etc.).
  4. Wait a few minutes. Vercel auto-issues an SSL certificate.
- 

## 6. Logs & debugging

---

When the production site misbehaves:

1. **Logs** tab → filter by route (e.g. `/api/revalidate` ).
2. You'll see every request with status code and any `console.log()` output from the route.
3. For build errors → **Deployments** tab → click the failed deployment → **Build Logs**.

Health check shortcut: `https://yoursite.com/api/check-setup` .

---

## 7. Limits & cost

---

The free **Hobby** plan covers small projects. Watch out for:

- 100 GB bandwidth / month
- 100 hours of build time / month
- 12 serverless function invocations per second
- Heavy images can blow through bandwidth — see `public/*.jpg` files (some are 7–20 MB; consider compressing)

Upgrade to **Pro** if traffic grows.

---

## 8. Team access

---

To invite a teammate:

1. **Settings** → **Team** (or your team's Vercel dashboard → Members).
  2. Invite by email.
  3. Choose role:
    - **Member** — can deploy and view logs
    - **Owner** — can change billing and domains
-

## 9. Common issues

---

| Symptom                             | Likely fix                                                                                                                                         |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Build fails with "Module not found" | A dependency was added locally but not committed. Run <code>pnpm install</code> and commit <code>package.json</code> + <code>pnpm-lock.yaml</code> |
| Build fails with "Type error"       | TypeScript error — fix in code, push again. Tip: run <code>pnpm build</code> locally before pushing                                                |
| Site shows but content empty        | Env vars not set in Vercel. Check <b>Settings → Environment Variables</b>                                                                          |
| Webhook returns 401 in Sanity logs  | <code>SANITY_REVALIDATE_SECRET</code> mismatch. Update Vercel and redeploy                                                                         |
| Images don't load                   | Sanity CORS doesn't include the Vercel URL. Add it in Sanity Manage                                                                                |
| Custom domain not resolving         | DNS not propagated yet. Wait, or check <code>dig</code>                                                                                            |

---

Next: [06 — Git & GitHub](#).

## 06 — Git & GitHub

Git tracks every change to the code. GitHub stores the code in the cloud and lets the team collaborate.

### 1. The repository

| Field          | Value                                                                                     |
|----------------|-------------------------------------------------------------------------------------------|
| Owner          | CrestAiLab (GitHub organisation)                                                          |
| Repo           | AIFirst                                                                                   |
| URL            | <a href="https://github.com/CrestAiLab/AIFirst">https://github.com/CrestAiLab/AIFirst</a> |
| Default branch | main (production)                                                                         |
| Other branches | renovate/* (created automatically by the Renovate bot to update dependencies)             |

To get access, ask an admin to invite your GitHub account to the **CrestAiLab** organisation.

### 2. The 15-second mental model

```
Your laptop      GitHub      Vercel
  _____      _____      _____
git commit —push→ main branch → auto-deploy → live site
```

You make changes locally → save them as a **commit** → **push** to GitHub → Vercel notices and deploys.

### 3. The commands you'll use 95% of the time

| Command                                       | What it does                                                     |
|-----------------------------------------------|------------------------------------------------------------------|
| <code>git status</code>                       | What's changed locally (your safety check before doing anything) |
| <code>git pull</code>                         | Download the latest commits from GitHub into your branch         |
| <code>git checkout -b feature/my-thing</code> | Create and switch to a new branch                                |
| <code>git add .</code>                        | Stage all changed files for the next commit                      |
| <code>git add path/to/file</code>             | Stage one specific file                                          |
| <code>git commit -m "describe change"</code>  | Save the staged changes as a commit                              |
| <code>git push</code>                         | Upload your commits to GitHub                                    |
| <code>git checkout main</code>                | Switch back to the main branch                                   |
| <code>git log --oneline -10</code>            | See the last 10 commits                                          |

### 4. The standard workflow for a change

Never push directly to `main` for a non-trivial change. Use a branch + Pull Request.

```
# 1. Make sure you're up-to-date
git checkout main
git pull

# 2. Create a branch for your change
git checkout -b feature/add-about-page

# 3. Make your edits, then save as a commit
git status           # see what changed
git add .            # stage everything (or pick specific files)
git commit -m "Add about page with team bios"

# 4. Push the branch to GitHub
git push -u origin feature/add-about-page
```

Then open a **Pull Request** on GitHub:

1. Go to the repo on GitHub. You'll see a yellow banner: "Compare & pull request" → click it.
2. Write a short title and description (what & why).
3. Request a reviewer.
4. Vercel will post a **preview URL** in the PR — share it for testing.

5. After approval, click **Merge pull request**.

Once merged, your branch is in `main`, and Vercel auto-deploys to production.

---

## 5. Things you should never do

---

- **✗ Don't commit `.env.local`** — it has secrets. (It's already in `.gitignore`, but double-check before `git add .`)
- **✗ Don't `git push --force` on `main`** — it rewrites history for everyone. If you really need a force push, do it on your own branch only.
- **✗ Don't commit `node_modules/` or `.next/`** — these are build artefacts, not source. Already gitignored.
- **✗ Don't commit huge binary files** — keep large images outside the repo if possible.

---

## 6. Reading commit history

---

```
git log --oneline -20      # last 20 commits, one line each
git log --stat             # commits with files changed and counts
git show <commit-hash>    # full diff of a specific commit
git blame path/to/file     # who last changed each line
```

In VS Code, **GitLens** shows blame info inline.

---

## 7. Undoing things safely

---

| You want to...                                           | Run                                            |
|----------------------------------------------------------|------------------------------------------------|
| Discard unstaged changes to a file                       | <code>git checkout -- path/to/file</code>      |
| Unstage a file (you ran <code>git add</code> by mistake) | <code>git restore --staged path/to/file</code> |
| Undo your last commit (keep the changes locally)         | <code>git reset --soft HEAD~1</code>           |
| Restore a file from main                                 | <code>git checkout main -- path/to/file</code> |
| Find a lost commit                                       | <code>git reflog</code>                        |

When in doubt, **commit before experimenting**. Commits are cheap; you can always rewind.

---

## 8. The `.gitignore` file

---

Already configured to skip:

- `node_modules/` (downloaded packages)
- `.next/` (build output)
- `.env*.local` (secrets)
- `.DS_Store` (macOS metadata)
- `tsconfig.tsbuildinfo` (TS incremental cache)
- `pnpm-store/`

If you're ever unsure whether a file should be committed, ask the team.

---

## 9. SSH vs HTTPS for cloning

---

- **SSH** ( `git@github.com:CrestAiLab/AIFirst.git` ) — set up an SSH key once, no password prompts.
- **HTTPS** ( `https://github.com/CrestAiLab/AIFirst.git` ) — easier to start; needs a Personal Access Token (PAT) when pushing.

Setup guide: GitHub Docs → "Adding a new SSH key to your GitHub account".

---

## 10. The Renovate bot

---

You'll occasionally see PRs from `renovate[bot]` proposing dependency upgrades. To handle one:

1. Open the PR — it lists what changed (e.g. `next: 15.0.3 → 15.0.4` ).
2. Wait for the Vercel preview to build.
3. Visit the preview URL → click around, make sure nothing is broken.
4. Merge if it looks good. The bot squashes its own commits and keeps things tidy.

If a Renovate PR breaks the build or preview, close the PR and the bot will retry on the next schedule.

---

## 11. Quick reference: solo branch workflow on a small team

---

```
git checkout main && git pull           # start fresh
git checkout -b fix/typo-in-hero        # branch off main
# ...edit files...
git add . && git commit -m "Fix typo in hero copy"
git push -u origin fix/typo-in-hero    # push to GitHub
# open PR, get review, merge
git checkout main && git pull           # local main is now up-to-date
git branch -d fix/typo-in-hero          # delete the local branch
```

---

Next: [07 — CI/CD](#).

# 07 — CI/CD

CI/CD = **Continuous Integration / Continuous Deployment**: the robots that build and ship our site automatically when something changes.

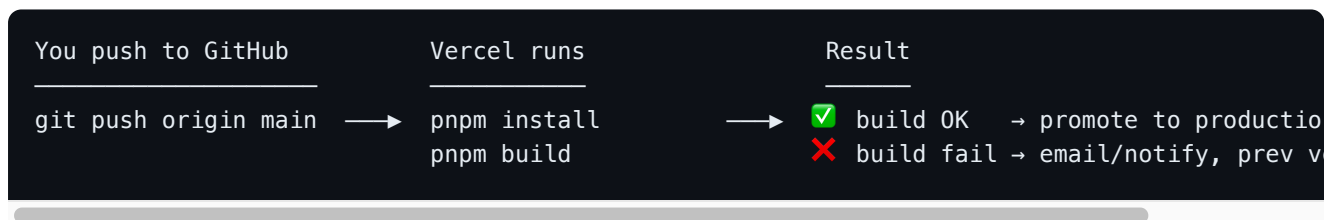
## 1. The short version

We have **two** automated pipelines:

| Pipeline                  | Trigger                    | Result                                                                 |
|---------------------------|----------------------------|------------------------------------------------------------------------|
| Code → Live site          | Push to GitHub             | Vercel rebuilds and deploys                                            |
| Content → Refreshed pages | Editor publishes in Sanity | Webhook hits <code>/api/revalidate</code> , Next.js drops cached pages |

There is **no GitHub Actions workflow yet**. All the CI/CD is provided by Vercel + the Sanity webhook.

## 2. The code pipeline (Git → Vercel)



### Per-branch behaviour

| Branch           | What Vercel does                                                                           |
|------------------|--------------------------------------------------------------------------------------------|
| main             | Build → deploy to <b>production</b> URL                                                    |
| Any other branch | Build → publish a <b>preview</b> URL (e.g. <code>aifirst-git-feature-x.vercel.app</code> ) |
| Pull Request     | Same as branch above; Vercel comments the preview URL on the PR                            |

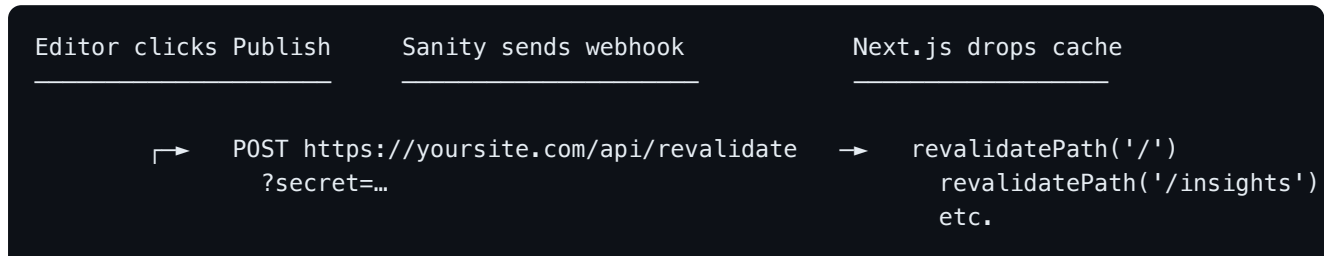
### What "build" actually means

1. Vercel checks out your commit.
2. Reads `vercel.json` → runs `pnpm install`.
3. Runs `pnpm run build` (which is `next build`).
4. If the exit code is 0, Vercel uploads the `.next/` output and switches the URL to it.

5. If non-zero, Vercel keeps the previous deployment live.

You can watch the build live: **Vercel** → **Deployments** → **click the running deployment** → **Build Logs**.

### 3. The content pipeline (Sanity → Vercel)



If the webhook isn't configured, content still updates eventually because Next.js caches expire on a timer — but it can take hours instead of seconds.

How to set up the webhook: see [04 — Sanity §6](#).

### 4. The Renovate dependency bot

Renovate is a third-party bot that watches `package.json` and opens PRs to update outdated libraries (e.g. `next: 15.0.3 → 15.0.4`).

| What you see                         | What to do                                                                         |
|--------------------------------------|------------------------------------------------------------------------------------|
| A PR from <code>renovate[bot]</code> | Wait for Vercel preview, click around, merge if OK                                 |
| Many PRs at once                     | Renovate batches by ecosystem; review them in order, easiest first                 |
| A PR fails the Vercel build          | Close it. Renovate will retry on its schedule, or you can investigate the breakage |

The `renovate` branch you may see in `git branch -a` is created by the bot.

### 5. There is no separate test suite

The codebase has no automated tests right now. The "test" before deploy is:

1. `pnpm build` succeeds locally (TypeScript + ESLint).
2. The Vercel preview URL works when you click around.
3. Code review on the PR catches obvious mistakes.

If you add tests later, the natural place is a GitHub Action — see §6.



## 6. Adding a GitHub Action (optional, future)

---

If you want to enforce checks **before** Vercel even tries to build (e.g. linting, type-checking, tests), add a workflow file:

```
# .github/workflows/ci.yml
name: CI

on:
  pull_request:
    branches: [ main ]
  push:
    branches: [ main ]

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v3
        with:
          version: 9
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'pnpm'
      - run: pnpm install --frozen-lockfile
      - run: pnpm lint
      - run: pnpm build
    env:
      NEXT_PUBLIC_SANITY_PROJECT_ID: ${ secrets.NEXT_PUBLIC_SANITY_PROJECT_ID }
      NEXT_PUBLIC_SANITY_DATASET: production
```

Place this at `.github/workflows/ci.yml`. GitHub will pick it up automatically. Add the relevant secrets in **GitHub → Settings → Secrets and variables → Actions**.

This is **not currently required** — Vercel covers the same build step.

---

## 7. Health checks after a deploy

---

After every production deploy, do this 1-minute smoke test:

1. Open `https://yoursite.com/` — homepage loads, sections show.
2. Open `https://yoursite.com/insights` — list shows.
3. Open `https://yoursite.com/api/check-setup` — JSON says all env vars set, Sanity reachable.
4. Edit any insight in `/studio` → publish → wait 5s → reload — change is visible.

If any step fails, check **Vercel → Logs** and **Sanity → Webhooks → recent deliveries**.

---

## 8. Rollback

---

The fastest rollback uses Vercel directly:

1. Vercel → **Deployments** tab.
2. Find the last good production deployment.
3. → **Promote to Production**.

That's it — no Git revert needed. Then fix the bug at your leisure.

---

## 9. CI/CD failure modes & fixes

---

| Symptom                                       | Likely cause                                                | Fix                                                                                 |
|-----------------------------------------------|-------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Vercel build fails:<br>"Cannot find module X" | New dependency not committed                                | Commit <code>package.json</code> and <code>pnpm-lock.yaml</code>                    |
| Vercel build fails:<br>TypeScript error       | Code error not caught locally                               | Reproduce with <code>pnpm build</code> locally, fix, push                           |
| Build succeeds but<br>page is blank           | Env var missing in Vercel                                   | Add it in <b>Settings → Environment Variables</b> → redeploy                        |
| Sanity publish doesn't<br>refresh site        | Webhook not firing or wrong secret                          | Sanity Manage → Webhooks → check delivery log; verify secret matches Vercel env var |
| Preview URL works,<br>production broken       | Different env vars between<br>Production and Preview scopes | Compare in Vercel <b>Settings → Environment Variables</b>                           |

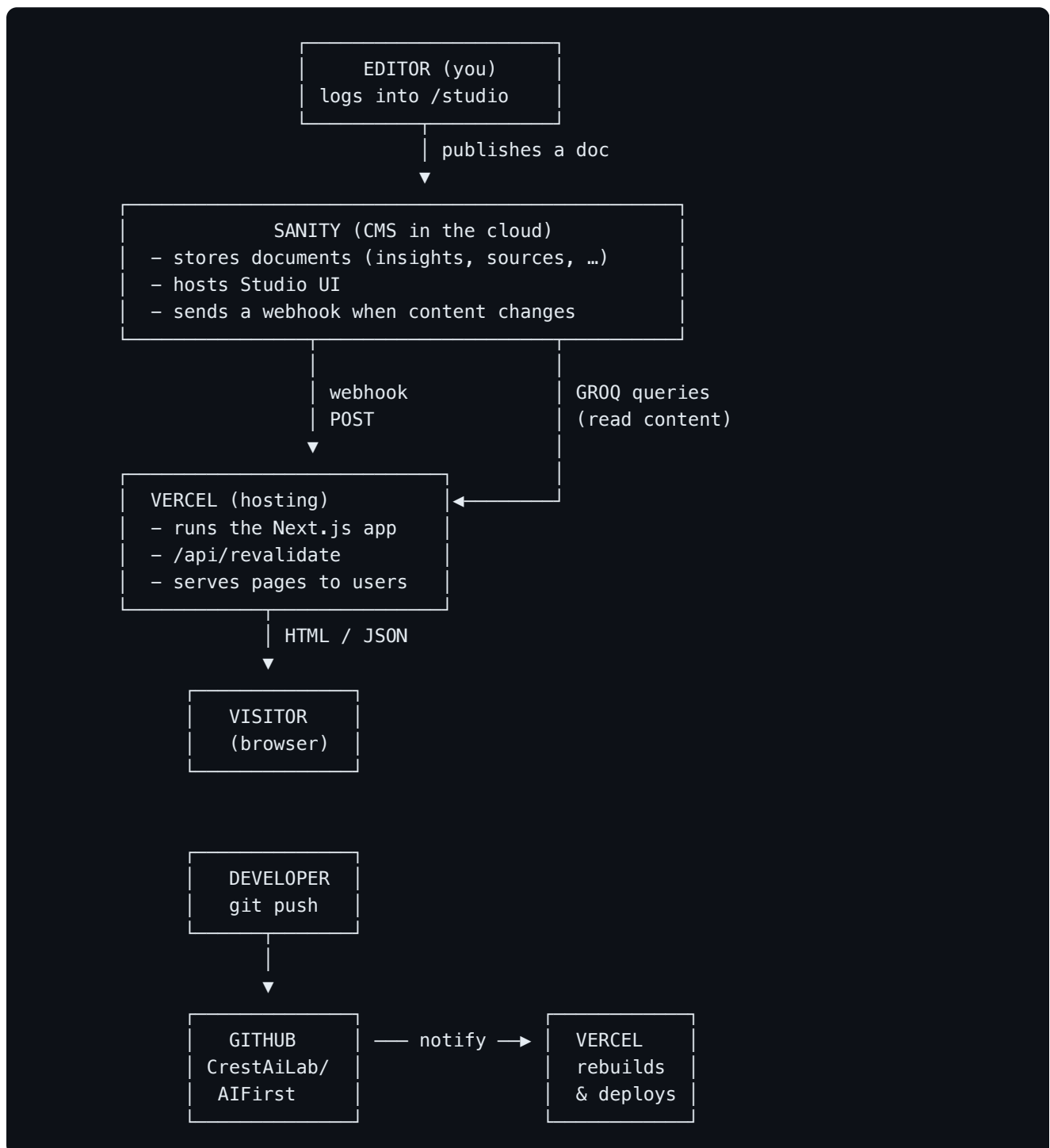
---

Next: [08 — How They Link Together](#) — the architecture diagram.

# 08 — How They Link Together

A picture of the whole system. Read this once and you'll always know "where does that thing live?".

## 1. The big diagram



---

## 2. Service-by-service: who talks to whom

---

| From                         | To                       | What they exchange                                       |
|------------------------------|--------------------------|----------------------------------------------------------|
| Visitor's browser            | Vercel                   | HTTP requests for HTML pages, images, API responses      |
| Vercel (Next.js server)      | Sanity (Content Lake)    | GROQ queries to fetch content (insights, sections, etc.) |
| Editor's browser             | Vercel → /studio         | Loads the Studio interface from our site                 |
| Editor's browser (in Studio) | Sanity (Content Lake)    | Reads/writes documents directly via Sanity's API         |
| Sanity                       | Vercel → /api/revalidate | Webhook POST when a document changes                     |
| Developer's laptop           | GitHub                   | <code>git push</code> uploads commits                    |
| GitHub                       | Vercel                   | Notifies Vercel of new commits → triggers build          |
| Vercel                       | GitHub                   | Posts deployment status / preview URL on PRs             |
| Renovate bot                 | GitHub                   | Opens PRs to update dependencies                         |

---

## 3. Where each "thing" actually lives

---

| Thing                                 | Lives in                                                               | How to access                                                               |
|---------------------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| The website code                      | GitHub: <code>CrestAiLab/AIFirst</code>                                | <code>git clone ...</code>                                                  |
| The website itself (deployed)         | Vercel                                                                 | Production URL (e.g. <code>aifirst.vercel.app</code> or your custom domain) |
| Content (insights, sources, sections) | Sanity Content Lake                                                    | <code>/studio</code> to edit, GROQ to read                                  |
| Studio (the editor UI)                | Inside our Next.js app                                                 | Visit <code>/studio</code> on whichever site is running the code            |
| Static images (decoration, logos)     | <code>public/</code> folder in the repo                                | Served by Vercel from <code>/icon.svg</code> etc.                           |
| Editor-uploaded images                | Sanity's CDN                                                           | <code>cdn.sanity.io</code> URLs (built via <code>urlFor()</code> )          |
| Secrets (API tokens)                  | Vercel env vars (production) and <code>.env.local</code> (your laptop) | Never in Git                                                                |

---

## 4. The three flows that explain everything

---

### Flow A — A visitor opens the homepage

1. Browser → `https://oursite.com/` → Vercel.
2. Vercel runs the Next.js homepage ( `app/page.tsx` ).
3. Inside the page, server code calls `client.fetch(pageContentQuery)` → Sanity.
4. Sanity returns the page sections.
5. Each section is rendered by `section-renderer.tsx` into HTML.
6. Vercel sends the HTML to the browser.
7. Browser shows the page.

If Sanity is unreachable, step 3 fails silently and the page falls back to `getDefaultSections()` from `lib/defaultSections.ts` — the site never goes blank.

### Flow B — An editor publishes a new article

1. Editor opens `/studio` → fills in fields → clicks **Publish**.
2. Studio writes the document to Sanity Content Lake.
3. Sanity sees there's a webhook configured → fires `POST https://oursite.com/api/revalidate?secret=...`.
4. Our `/api/revalidate` route validates the secret, reads which document type changed, and calls `revalidatePath()` for the affected pages.
5. Next.js drops those pages from its cache.
6. Next visitor → Flow A re-runs and fetches the new content.

### Flow C — A developer ships a code change

1. Developer creates a branch, edits files, commits, pushes to GitHub.
  2. Developer opens a Pull Request.
  3. Vercel sees the new branch → builds it → posts a **preview URL** in the PR.
  4. Reviewer clicks the preview, sees the change, approves.
  5. PR is merged into `main`.
  6. Vercel sees the new `main` commit → builds → if green, switches the production URL to it.
  7. Old version stays available in **Deployments** for instant rollback.
-

## 5. Where each environment variable is read

| Env var                                    | Read by                                                                                      | Used for                                                  |
|--------------------------------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <code>NEXT_PUBLIC_SANITY_PROJECT_ID</code> | Browser + Server<br>( <code>lib/sanity/client.ts</code> ,<br><code>sanity.config.ts</code> ) | Telling the Sanity client<br>which project to talk to     |
| <code>NEXT_PUBLIC_SANITY_DATASET</code>    | Browser + Server                                                                             | Same — which dataset<br>(always <code>production</code> ) |
| <code>SANITY_API_TOKEN</code>              | Server only (scripts, server<br>components if writing)                                       | Authenticated<br>reads/writes                             |
| <code>SANITY_REVALIDATE_SECRET</code>      | Server only ( <code>/api/revalidate</code> )                                                 | Validating that a<br>webhook is genuinely<br>from Sanity  |

`NEXT_PUBLIC_*` values are *baked into* the JavaScript that ships to the browser. Don't put secrets there.

## 6. What can go wrong, where to look

| Symptom                                                  | First place to check                                                                        |
|----------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Site is down                                             | Vercel → Deployments (was the last build red?)                                              |
| Site is up but content is<br>wrong/stale                 | Sanity → Webhooks → recent deliveries, then <code>/api/check-setup</code>                   |
| Studio ( <code>/studio</code> ) shows blank<br>or errors | Sanity → Manage → CORS Origins (is your domain listed?)                                     |
| Build is failing                                         | Vercel → Deployments → click the failing build → Build Logs                                 |
| Editor can't log into Studio                             | Sanity → Manage → Members (are they invited?)                                               |
| Image links broken                                       | Sanity → Manage → CORS, or check <code>next.config.js</code><br><code>remotePatterns</code> |
| Secret leaked                                            | Sanity → Manage → API → Tokens → revoke + create new + update<br>Vercel + redeploy          |

## 7. Mental shortcut

Whenever you're confused, ask: **"Is this about code, content, hosting, or version control?"**

| If...                                | ...go to                                                                                                      |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------|
| The behaviour or layout is wrong     | <b>Code</b> → GitHub repo, edit files, push                                                                   |
| The text or images are wrong         | <b>Content</b> → Sanity Studio                                                                                |
| The site is down or env vars are off | <b>Hosting</b> → Vercel dashboard                                                                             |
| You need history or to roll back     | <b>Version control</b> → Git / GitHub for code, Sanity timeline for content, Vercel deployments for live site |

---

Next: [09 — How To Deploy](#) for the deploy walkthrough.

# 09 — How To Deploy

---

Two flavours: **(A)** routine deploy of an existing site, and **(B)** first-time deploy from scratch.

---

## A. Routine deploy (the common case)

---

You have an existing Vercel + Sanity setup and you want to ship a code change.

```
# 1. Make sure your local main is up-to-date
git checkout main
git pull

# 2. Branch off
git checkout -b feature/short-description

# 3. Edit code. Verify locally.
pnpm dev           # check in browser
pnpm build         # ensure it builds clean

# 4. Commit and push
git add .
git commit -m "Add short description"
git push -u origin feature/short-description
```

Then on GitHub:

1. Open the PR (a banner appears after pushing).
2. Wait ~1 minute for the **Vercel preview URL** to appear in a comment.
3. Click the preview, click around, verify your change works.
4. Get a review (or self-merge if you're solo).
5. **Merge pull request** → choose **Squash and merge** (cleanest history).
6. Vercel auto-deploys to production within ~2 minutes.

### Smoke-test after deploy

Open these in order:

- <https://yoursite.com/> — homepage loads, sections render.
- <https://yoursite.com/insights> — list shows.
- <https://yoursite.com/api/check-setup> — JSON shows everything green.

If anything is red, **rollback first, debug second** (Vercel → Deployments → → Promote previous to Production).

---



## B. First-time deploy from scratch

---

Use this if you're setting up a brand-new copy of the site (new Vercel project, new Sanity dataset).

### Step 1 — Create the Sanity project

1. Sign in at [sanity.io/manage](https://sanity.io/manage).
2. **Create new project**. Pick a name. Choose dataset name `production`.
3. Note the **Project ID** (you'll need it next).
4. **API → CORS Origins** → add `http://localhost:3000` and your future production URL. Tick **Allow credentials**.
5. **API → Tokens** → create a token named "server" with **Editor** permission (only if you'll run write scripts). Copy and save the token — you can't view it again.

### Step 2 — Push the code to GitHub

1. Create an empty repo at github.com under your org.
2. From your local copy:

```
git remote set-url origin git@github.com:YOUR_ORG/your-repo.git
git push -u origin main
```

### Step 3 — Connect Vercel

1. [vercel.com](https://vercel.com) → **Add New Project** → Import the GitHub repo.
2. Framework: Next.js (auto-detected).
3. Build & install commands: leave defaults — `vercel.json` already says `pnpm install / pnpm run build`.
4. **Environment Variables** — add:
  - `NEXT_PUBLIC_SANITY_PROJECT_ID` = <from step 1>
  - `NEXT_PUBLIC_SANITY_DATASET` = `production`
  - `SANITY_API_TOKEN` = <token from step 1> (optional)
  - `SANITY_REVALIDATE_SECRET` = output of `openssl rand -hex 32`
5. Click **Deploy**. Wait for the first build to finish.

You now have a live URL like `your-repo.vercel.app`.

### Step 4 — Configure the Sanity webhook (so publishes refresh the site)

1. [sanity.io/manage](https://sanity.io/manage) → your project → **API → Webhooks → Create webhook**.
2. **Name**: Vercel revalidate
3. **URL**: `https://your-repo.vercel.app/api/revalidate?secret=<the same secret>`
4. **Trigger on**: Create, Update, Delete

5. **Filter:** leave blank (or `_type in ["pageContent","insight","source","communityPost"]` )
6. **Projection:** `{ _type, _id, slug }`
7. **HTTP method:** POST
8. Save.

Test it: open `https://your-repo.vercel.app/api/revalidate?secret=<the secret>` in a browser — you should see `{"revalidated":true,...}` .

## Step 5 — Seed the homepage

On your laptop, with `.env.local` filled in:

```
pnpm seed:sanity
```

This creates the `pageContent` document with the default sections. You can now visit `/studio` and customise.

## Step 6 — Add your custom domain (optional)

1. Vercel → **Settings** → **Domains** → **Add** → type your domain.
2. Vercel shows DNS records. Add them at your registrar.
3. Wait 5–30 minutes for DNS + SSL.
4. Update Sanity CORS to include the new domain.
5. Update the Sanity webhook URL to use the new domain (or keep the `vercel.app` URL — both work).

## Step 7 — Final verification

- Open `https://your-domain/api/check-setup` — all green.
- Open `/studio` , log in, edit something, publish, watch the homepage update.
- Open the live URL on your phone to confirm responsive layout.

---

## C. Hot-fixing a production bug

When something is on fire and you need to ship a fix immediately:

```
git checkout main
git pull
git checkout -b hotfix/short-name
# ...edit...
pnpm build # must succeed locally
git add . && git commit -m "Hotfix: ..."
git push -u origin hotfix/short-name
```

Open PR → if you're confident, merge straight away (skip review). Vercel deploys in ~2 minutes.

If you can't wait for the build:

1. Vercel → Deployments → find the last-known-good production deployment.
2. → **Promote to Production** → instant rollback.
3. Then take your time fixing.

---

## D. Reverting a code change

---

If a merged PR caused the problem:

```
git checkout main && git pull
git revert <bad-commit-hash>      # creates a new commit that undoes it
git push
```

A new "Revert" commit goes through the normal pipeline → Vercel deploys it.

Alternative (faster): use Vercel's **Promote to Production** on the previous good deployment — no Git change needed.

---

## E. Pre-deploy checklist (before merging)

---

- ☐ `pnpm build` succeeds locally
- ☐ `pnpm lint` passes (or warnings are intentional)
- ☐ Vercel preview URL works in a browser
- ☐ No new env var is required (or it's been added in Vercel for **both** Production and Preview scopes)
- ☐ No `console.log` debugging left behind
- ☐ No `.env*` file committed by accident
- ☐ Description on the PR explains the *why*, not just the *what*

---

Next: [10 — How To Develop A New Feature](#) — concrete recipes.

# 10 — How To Develop A New Feature

---

Recipes for the most common kinds of changes. Each recipe ends with "ship it" — meaning push the branch, open a PR, merge.

---

## Recipe 1 — Change copy or styling on the homepage (no code)

---

If the change is just text, an image, or reordering sections — **don't touch code**, do it in Sanity.

1. Open `/studio` .
2. **Page Content** → edit fields, drag sections, toggle visibility.
3. **Publish**.
4. Wait ~5 seconds → homepage updates.

If the change is a Tailwind class tweak (e.g. spacing, colour) — that's a code change; see Recipe 6.

---

## Recipe 2 — Add a new homepage section type

---

Goal: introduce a new section like "Testimonials" that editors can add in Sanity.

1. **Add a frontend component** at `components/testimonials.tsx` :

```
type Testimonial = { quote: string; author: string }
type Props = { heading: string; items: Testimonial[] }

export function Testimonials({ heading, items }: Props) {
  return (
    <section className="py-16">
      <h2 className="text-3xl font-bold">{heading}</h2>
      <ul className="mt-6 grid md:grid-cols-3 gap-4">
        {items.map((t, i) => (
          <li key={i} className="rounded-lg border p-6">
            <blockquote className="italic">{t.quote}</blockquote>
            <p className="mt-2 text-sm">— {t.author}</p>
          </li>
        ))}
      </ul>
    </section>
  )
}
```

2. **Wire it into** `section-renderer.tsx` : import the component and add a case for the new `sectionType` .
3. **Update the Sanity schema** at `sanity/schemaTypes/pageSection.ts` :

- Add `'testimonials'` to the list of allowed `sectionType` values.
  - Add fields ( `heading` , `items` array of `{quote, author}` ) and conditionally show them only when `sectionType === 'testimonials'` .
4. **Update the GROQ query** in `lib/sanity/queries.ts` so the new fields are returned.
  5. **Update the type** in `lib/sanity/types.ts` to include the new section shape.
  6. Run `pnpm dev` , open `/studio` , add a Testimonials section, publish, verify it appears.
  7. Ship it.
- 

## Recipe 3 — Add a new field to an existing document

---

Example: add an "estimated reading time" field to insights.

1. Edit `sanity/schemaTypes/insight.ts` and add:

```
defineField({
  name: 'readingTime',
  title: 'Reading time (minutes)',
  type: 'number',
}),
```

2. Reload `/studio` — the new field appears.
3. Update `lib/sanity/queries.ts` queries that fetch insights to include `readingTime` .
4. Update `lib/sanity/types.ts` `Insight` type to add `readingTime?: number` .
5. Render it in `app/insights/[slug]/page.tsx` :

```
{insight.readingTime && <span>{insight.readingTime} min read</span>}
```

6. Ship it. Backfill old insights via Studio if needed.
- 

## Recipe 4 — Add a new page

---

Example: `/about` .

1. Create `app/about/page.tsx` :

```
export const metadata = { title: 'About – AIFirst' }

export default function AboutPage() {
  return (
    <main className="container mx-auto px-4 py-16">
      <h1 className="text-4xl font-bold">About</h1>
      <p className="mt-4 text-white/70">...</p>
    </main>
  )
}
```

2. Optional: add a link in `components/header.tsx` .
3. Visit `http://localhost:3000/about` to verify.
4. Ship it.

---

## Recipe 5 — Add a new API route

---

Example: `/api/health` returns `{ ok: true }` .

1. Create `app/api/health/route.ts` :

```
import { NextResponse } from 'next/server'

export async function GET() {
  return NextResponse.json({ ok: true, time: new Date().toISOString() })
}
```

2. Visit `http://localhost:3000/api/health` .
3. Ship it.

For a route that talks to Sanity, import `client` from `@/lib/sanity/client` and use `client.fetch(...)` .

---

## Recipe 6 — Style change

---

Find the component, add/remove Tailwind classes, save, reload.

If the same class repeats everywhere, consider extracting a helper or component. If it's project-wide (e.g. font, base colour), edit `app/globals.css` or `tailwind.config.ts` .

Tip: use the **Tailwind CSS IntelliSense** VS Code extension for autocomplete.

---

## Recipe 7 — Add an environment variable

---

1. Add it to `.env.local.example` (with an empty value and a comment explaining what it's for).
  2. Add it to your `.env.local` with the real value.
  3. Add it in Vercel → **Settings** → **Environment Variables** for **Production** AND **Preview**.
  4. Reference it in code: `process.env.MY_VAR` .
    - If it must reach the browser, prefix it `NEXT_PUBLIC_MY_VAR` .
  5. Redeploy (push any commit) so Vercel picks it up.
- 

## Recipe 8 — Adding a shadcn/ui component

---

```
pnpm dlx shadcn@latest add dialog
```

This adds `components/ui/dialog.tsx` . Import and use it like the existing `Button` , `Card` , etc.

---

## Recipe 9 — Update a dependency

---

If Renovate hasn't proposed it yet:

```
pnpm up next                # latest minor/patch
pnpm up next@15.1          # specific version
pnpm up --latest            # update everything to latest (risky)
```

Always run `pnpm build` and click around the dev server before committing.

---

## Recipe 10 — Debugging "the page won't update"

---

1. **Locally:** restart `pnpm dev` (sometimes the cache is sticky).
  2. **In production:**
    - Visit `/api/check-setup` — env vars OK?
    - Sanity → Manage → Webhooks → recent deliveries — are they 2xx?
    - Manually fire: `https://yoursite.com/api/revalidate?secret=...` — does it return `{revalidated:true}` ?
    - Vercel Logs — any error from `/api/revalidate` ?
-

## Recipe 11 — Testing a Sanity change before publishing

---

Sanity has **drafts** built in. **Save** without publishing → only Studio sees it. The public site only reads published documents.

If you want a "draft preview" on the website itself, you'd need to add Sanity's preview mode (not currently set up). Ask the team before adding this — it's a meaningful feature.

---

## Recipe 12 — Removing something safely

---

To delete a feature:

1. Remove the component file and any references.
2. Remove the GROQ query and the type if nothing else uses them.
3. If a Sanity field is being dropped, **don't delete it from the schema right away** — first deploy the code that ignores it, then remove the schema field, then optionally clean up old document data.

This avoids "the field exists in Studio but the code crashes" or vice versa.

---

## Universal "ship it" checklist

---

Before opening a PR:

- ☐ `pnpm dev` works, change is visible
- ☐ `pnpm build` finishes with no errors
- ☐ No secrets committed
- ☐ No `console.log` debugging left
- ☐ PR description explains *why*, not just *what*
- ☐ If a new env var was added, it's added to **both** Vercel scopes

After merging:

- ☐ Vercel deployment is green
- ☐ Production site loads and behaves as expected
- ☐ `/api/check-setup` is green